Security & Architecture Review: ai-powered-soc-lab

Repository: github.com/vitaliibalint39-oss/ai-powered-soc-lab Review date: October 10, 2026

Scope and limitations

This review is based only on the repository landing page and README. Attempts to fetch `soc_assistant.py`, `requirements.txt`, `docs/` and `examples/` failed (repeated 503 errors; the `docs/` tree is blocked to automated access). **No source code was reviewed, so this document contains no line-level bug findings.** Items under "Things to verify in the code" are checks to run, not confirmed defects.

What the repo is

The README describes a proof of concept. A Wazuh agent on `ubuntu-soc` sends alerts to a Wazuh manager. A Python script pulls alerts from the Indexer over an SSH tunnel, sends them to Qwen 3.5 4B in LM Studio, and writes Markdown reports. The repo has 2 commits, 0 stars and 0 forks. The README states the scope plainly: manual batch processing, no autonomous response, analyst review required. No overclaiming was found.

Strengths (as documented)

- **Network design:** The Indexer is bound to loopback and reached through an SSH local forward, so port 9200 is not exposed to the VM network.
- **TLS:** The README says verification stays on with a CA file, and tells users to fix the certificate SAN rather than disable verification.
- **Least privilege:** A `soc_reader` account with a custom role is used, and a 403 on the security admin endpoint was tested.
- **Honest threat model:** Log content is treated as untrusted, model output as advisory, and no remote execution or auto-blocking is allowed.
- **Hygiene:** `root-ca.pem`, state, reports and `.venv` are marked local-only.

Risks and gaps visible from the README

High priority

1. **Prompt injection through log content.** The README says logs are untrusted, but not how that is enforced. Attacker-controlled fields (usernames in SSH failures, URLs, command lines, HTTP user agents) flow into an LLM prompt, and a 4B model is easy to manipulate. The no-remote-action design limits the damage, but a poisoned log can still suppress or falsify a triage report. Check whether alert data is delimited, truncated and kept separate from instructions, and whether model output is validated against a schema.
2. **Silent alert loss.** Only the latest 20 alerts at level 5 or higher are fetched for one hard-coded agent. A burst of more than 20 alerts between runs drops the older ones with no warning. For a security tool, silent loss is worse than an error. The README acknowledges the missing pagination, but it should be treated as the top functional gap.
3. **Non-atomic state.** The README admits that report writing and `processed_events.json` updates are not transactional. A crash after the report but before the state save causes duplicates; the reverse loses an alert. Check for atomic writes (temp file plus rename) and handling of a half-written JSON file.
4. **Truncated model output is accepted.** The README says output can hit the token limit. If truncated reports are saved and the ID is marked processed, those alerts are effectively closed with incomplete analysis. Check whether truncation is detected via `finish_reason` and retried.

Medium priority

5. **Credentials.** The password is prompted interactively, which suits a lab but blocks scheduling (a roadmap item). Make sure automation doesn't push users toward a plaintext password in the script or an env file. The quick start shows a named personal account for the tunnel; a dedicated, restricted tunnel user (`PermitOpen`, no shell) would be better.
6. **Sensitive data in reports and prompts.** Local inference avoids cloud exposure, but reports contain IPs, usernames and hostnames. Redaction is a TODO. If report filenames derive from alert fields rather than the alert ID, there is a path-injection risk.

7. **Hard-coded configuration.** The `agent.name` filter and connection constants live in the script rather than a config file or CLI arguments, so one script can't cover more than one agent.
8. **No evaluation of AI quality.** There is no ground truth or benchmark for how well the 4B model classifies alerts. The README correctly says Wazuh levels and model severity are not equivalent, but nothing measures disagreement or false-positive handling.
9. **Dependencies.** The stack lists only `requests`, a small attack surface. `requirements.txt` was not readable, so check for pinned versions and hashes, and run `pip-audit`.

Low priority and repo hygiene

- No tests, no CI (Actions exists but no workflows were visible), and no license is visible.
- The README mentions `docs/SETUP_UK.md`, `SECURITY.md` and `TEST_PLAN.md`; their contents were not verified.
- The README layout shows `AI-SOC-Lab/` while the repo is `ai-powered-soc-lab`, a minor inconsistency.
- Screenshots and the `examples/` report should be re-checked for internal IPs, hostnames and usernames. The README publishes lab IPs and the username `vitalii` in the tunnel command. These are private-range addresses, so the risk is small, but the username is a real identifier.
- The PowerShell example shows `$HOME.ssh`, which looks like a missing backslash and may break copy-paste.
- The "Confirm clean repeat-run behavior" roadmap item is unchecked, so the dedup logic is unverified by the author.

Things to verify in `soc_assistant.py`

- `requests` calls use `verify=` with the CA path, never `False`, and set `timeout=`.
- How the prompt is built, and whether alert fields are truncated and delimited.
- Whether `finish_reason`/truncation and malformed responses are handled, and what happens if LM Studio is down.
- Where the processed ID is saved relative to the report write.
- How the report filename is generated.
- Whether exceptions are caught broadly and swallowed.

- Whether the full alert, including `full_log`, goes to the model, and whether secrets in logs get through.

Prioritized roadmap

Phase 1: correctness and trust

1. Checkpointed, paginated ingestion using `search_after` on timestamp plus `_id`, with a visible warning when alerts are skipped.
2. Atomic state writes; mark an alert processed only after the report is fully written and validated.
3. Detect truncation and retry or flag the report as incomplete.
4. Prompt-injection hardening: structured delimiters, field allow-listing and length limits, and JSON-schema-constrained output.
5. Sanitise report filenames and add a redaction pass.

Phase 2: engineering baseline

6. Move configuration to a file or CLI arguments, and support multiple agents.
7. Add tests with sanitized fixtures, plus CI running `ruff`, `pytest`, `bandit` and `pip-audit`.
8. Pin dependencies and add a license.

Phase 3: AI quality

9. Build a small labelled alert set and measure model accuracy against it; compare Qwen sizes and prompt variants.
10. Add MITRE ATT&CK mapping and deduplication or grouping of related alerts. Output both machine-readable JSON and Markdown.
11. Add confidence and "insufficient data" outputs so the model can decline to classify.

Phase 4: operations

12. Scheduling (Task Scheduler or a service) with a secrets approach that avoids plaintext passwords, plus health logging so a silent failure is visible.

13. Optional: a Wazuh integration or webhook path to replace polling, and a read-only analyst UI. Keep automated response out of scope unless there is a human approval step.

Next step

Share `soc_assistant.py` and `requirements.txt` to convert the verification list into concrete findings with line references.